

Class 16: Modeling Fundamentals, Experiment Tracking, and Tuning

Data-driven Systems Engineering

Agenda

- Modeling as abstraction and representation choice.
- Matching model families to problem types and constraints.
- Why experimentation is engineering practice, not casual score-hunting.
- What must be tracked to compare models fairly.
- Tuning strategies and Neptune-style experiment logging in this repository.
- From a tuned run to a governed, traceable artifact.

Learning goals

- Frame modeling as an engineering choice among accuracy, interpretability, speed, and maintainability.
- Match model families to data, objectives, and operational constraints.
- Design an experiment before running it, and know what must be logged to compare runs fairly.
- Explain how hyperparameter search connects to reproducibility and later delivery.

Course Map and Running Cases



Today in the course

Focus: Choosing model families and turning experimentation into a disciplined, tracked practice.

Bridge: With features settled, this class asks how to compare modeling choices honestly, and how to keep evidence for why one model was selected over another.

Fraud case

In the fraud case, we compare model families and hyperparameters for a rare-event classifier and log every run so results stay reproducible.

Pasteurization case

In the pasteurization case, experiment tracking can compare forecasting variants or preprocessing choices without losing context between runs.

Course thread: A good model choice must be explainable, repeatable, and ready for later delivery, not just the best number in a notebook.

What is modeling?

- Modeling represents patterns in data so the system can generalize to unseen cases.
- Every model encodes assumptions about structure, noise, and decision boundaries.
- Choosing a model means choosing a trade-off among accuracy, interpretability, speed, and maintainability.

Useful modeling questions

1. What decision or prediction are we supporting?
2. What kind of target do we have: class, score, or continuous value?
3. What kinds of errors are expensive?
4. How fast must training and inference be?
5. How often do we expect the data distribution to change?

Model families in context

Linear models	Fast, simple, and often strong baselines.
Tree-based models	Good for heterogeneous tabular data and nonlinearity.
Probabilistic models	Useful when uncertainty and conditional structure matter.
Neural networks	Powerful for large unstructured or highly nonlinear problems.
Online models	Update incrementally as new observations arrive.

More realistic examples

- Fraud detection: random forest or gradient boosting with threshold tuning.
- Wine-quality-style regression: gradient boosting tuned by grid search and tracked in Neptune.
- Pasteurization forecasting: incremental regression, revisited fully in Class 18.
- Industrial anomaly detection: useful only if alert cost and false-positive rate are understood upfront.

Repository example: a baseline training pipeline

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)  
  
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

- From 'streamlit/modeling.py', mirrored in 'notebooks/Class6_Model_Training_.ipynb'.
- Stratification matters because fraud is rare; a naive split can distort the class balance a model actually sees during training.

Experimentation is an engineering discipline

- The purpose of experimentation is not to collect many scores, but to support a credible model choice.
- Every run should answer a question: does a new feature help, does a different model family help, or does tuning justify extra complexity?
- Fix the evaluation protocol and decide what counts as a meaningful improvement before looking at results.
- When the question is unclear, experiment tracking becomes a storage system for confusion rather than evidence.

What must be tracked, and why

Dataset version / extraction date

So a result can be reproduced against the same data later.

Feature pipeline version

So preprocessing changes are not confused with model changes.

Hyperparameters and seeds

So the exact configuration that produced a result is known.

Metrics, confusion matrices, artifacts

So the trained output, not just a number, can be audited.

Code commit or notebook snapshot

So the logic behind a run can be replayed months later.

Fair comparison versus common bias

A fair comparison has

- The same dataset split or protocol
- The same preprocessing, unless that is the experiment
- Metrics chosen for the real decision

Bias creeps in from

- Reusing the test set until it becomes part of tuning
- Mixing notebook state and hidden edits
- Reporting only the winning run

Model search strategies

Manual tuning	Small problems and pedagogical intuition.
Grid search	Small, well-structured search spaces.
Random search	Better coverage when many hyperparameters matter unevenly.
Bayesian optimization	Expensive models where each run has high cost.

- A tuned model should beat a simple, previously deployed, or default baseline by a meaningful margin, not just a decimal point.

Neptune example: defining a search space

```
model = GradientBoostingRegressor(random_state=42)

param_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.05, 0.1, 0.2],
    "min_samples_split": [2, 5, 10],
}
```

- From 'notebooks/Class7_Model_Tuning_Neptune.ipynb', which uses the Neptune.ai experiment tracker alongside 'GridSearchCV'.
- The grid gives a realistic trade-off among model complexity, fit time, and predictive gain.

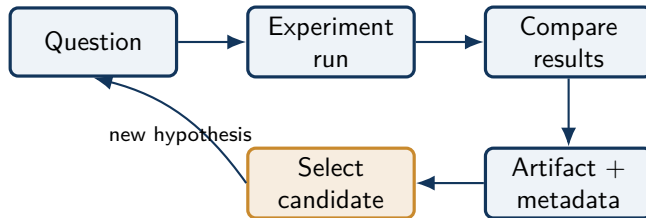
Neptune example: tuning and logging together

```
grid_search.fit(X_train, y_train)

run["best/parameters"] = grid_search.best_params_
run["best/cv_score"] = grid_search.best_score_
```

- The important idea is not Neptune specifically, but the discipline of storing parameters and metrics together, in the same place, automatically.
- That same pattern maps naturally onto MLflow, Weights & Biases, or an internal experiment registry.

From notebook run to governed asset



Neptune example: evaluate and persist the winner

```
best_model = grid_search.best_estimator_  
y_pred = best_model.predict(X_test)  
  
run["test/mse"] = mean_squared_error(y_test, y_pred)  
run["test/r2"] = r2_score(y_test, y_pred)  
  
joblib.dump(best_model, "best_gb_model.pkl")  
run["artifacts/model"].upload("best_gb_model.pkl")
```

- A useful experiment ends with a traceable artifact, not only a screenshot of the best score.
- This is also the natural bridge to model registries and deployment, covered fully in Class 20.

Common tuning mistakes

- Tuning on the test set.
- Comparing runs trained on different preprocessing logic.
- Reporting only the best metric, not its variability or computational cost.
- Forgetting that a slightly weaker model may be much easier to deploy, explain, or monitor.

Repo activity

- 'notebooks/Class6_Model_Training_.ipynb' gives the baseline split, tune, evaluate, and serialize workflow.
- 'notebooks/Class7_Model_Tuning_Neptune.ipynb' adds Neptune.ai tracking on top of the same grid-search pattern.
- Together they show the same discipline applied with and without an experiment tracker.

Question

Two runs report similar cross-validation scores but were trained on differently scaled inputs. Can we trust the comparison?

- This connects tracked metadata directly back to the “what must be tracked” table earlier in this class.

Papers and materials

Neptune documentation; MLflow documentation; scikit-learn model selection guide; Bergstra and Bengio (2012), Random Search; Bergstra et al. (2013), Hyperopt.

From This Class to the Next

What stays with us

- Model selection is tied to the prediction task, the error costs, and the constraints of the system it will live in.
- A useful experiment records data, configuration, metrics, and artifacts together, automatically.
- Hyperparameter search is valuable only when it is reproducible, comparable, and beats a real baseline.

Project use

Project teams should now be able to defend a baseline, compare tracked alternatives, and explain why a chosen model fits the fraud or pasteurization setting.

Next connection

Class 17 turns these tracked results into evaluation and decision-making: metrics that reflect the real task, and interfaces that turn a score into an action.

Course thread: Reproducible experimentation is what makes the next step, judging a model's real world